

Protocolos de comunicacion

Jaime Nazar Anchorena

Contents

Unidad 1: Introduccion	4
Modelo OSI - TCP/IP	5
Capa de enlaces	5
Capa de ruteo	6
Capa de transporte: Puestos	6
Capa de aplicacion: soporte a aplicaciones	6
Serializacion	6
Unidad 2: HTTP	6
HTTP 0.9(1991)	6
HTTP 1.0(1996)	7
HTTP 1.1(1999)	7
HTTP/2	7
HTTP/3	7
Recursos	7
URI(RFC 3986 3.3)	7
URL	7
URN	8
Request menos utilizados	8
Respuestas	8
Tipos de informacion	8
GET vs POST	9
Headers HTTP	9
Tipos	9
Headers generales	9
Headers de solicitud	9
Headers de respuesta	10
Headers de contenido	10
Proxy servers	10
Proxy reverso	10
Cookies	12
Curl	12
Resolucion de nombres - DNS	12
Como se consiguen los nombres	13

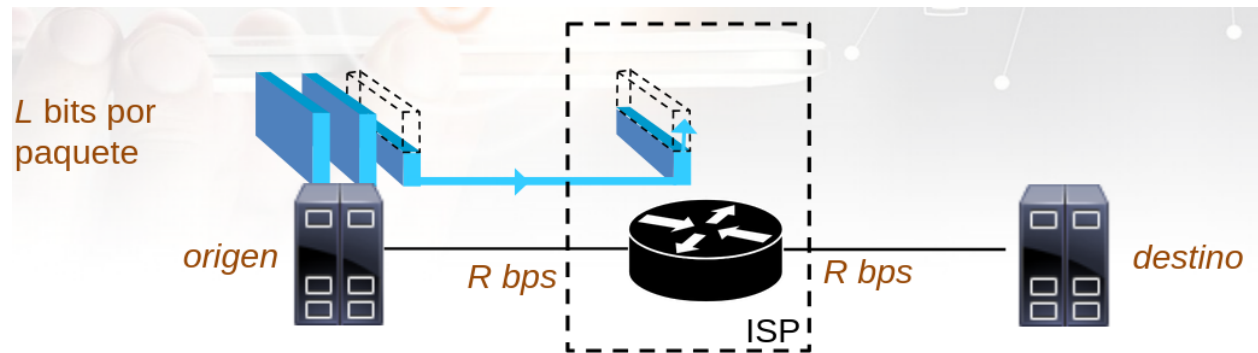
Cache	14
Configuracion de servidor DNS	14
Registros DNS	14
Formato RR	14
Split-horizon	15
DNS dinamico	15
DNS spoofing	15
Metodo: Enviar respuesta sin recibir pregunta	15
Metodo: DNS sniffer	15
Metodo: DNS cache poisoning	15
Notas practica	15
Unidad 4: Correo electronico	16
Envio de datos binarios	16
Como mandar mail con netcar	17
Protocolos de entrega final al usuario	17
Encoding: Base64	17
SPAM	18
Tecnicas para autentificar mails	18
SPF	18
DKIM	19
Unidad 5: TLS/SSL	19
SSL	19
Claves asimetricas	19
Tipos de SSL handshake	20
TLS	20
DNS over TLS(DoT)	20
DNS over HTTPS(DoH)	20
HTTPs y proxy HTTP	20
Intercepcion HTTPS con MITM	20
Deteccion	20
Limitaciones	20
Unidad 6: Protocolos de transporte	21
Multiplexacion	21
Demultiplexacion sin conexion	21
UDP	21
Demultiplexacion orientado a conexion	22
TCP	22
Repaso conexion confiable	22
Control de flujo	23
Control de congestion	23
NAPT (aka NAT)	24
SNAT	24
DNAT	24

Firewall	24
--------------------	----

Unidad 1: Introduccion

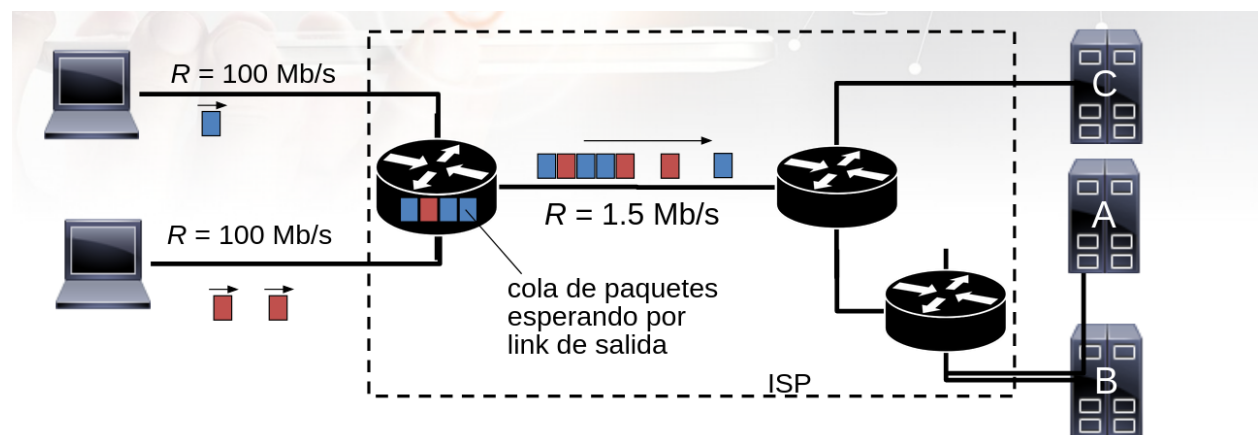
Las redes que vamos a ver es un conjunto interconectado de hosts, cada host es un equipo autonomo y no son necesariamente computadoras(pensar en IoT). El host envia datos en paquetes, de L bits de longitud.

Hay un problema, por ejemplo cuando contratamos un ISP, ya que nos venden la velocidad entre nuestra casa al ISP, pero no garantiza la misma velocidad a otros servicios. Ademas, si todos se conectan al mismo ISP, aparece un cuello de botella, haciendo que todo sea mas lento.



$$\text{Demora de transmisión de un paquete} = \text{tiempo necesario para transmitir paquete de } L \text{ bits} = \frac{L \text{ (bits)}}{R \text{ (bits/sec)}}$$

store and forward: el paquete debe arribar al router en forma completa antes de ser retransmitido



encolamiento y pérdida:

- ❖ Si la tasa del link de entrada es superior a la de transmisión en un determinado período:
 - se encolan paquetes, esperando por ser transmitidos
 - si el buffer está lleno, paquetes entrantes se "droppean"

Antes, tampoco habia estandares, entonces sistemas de diferentes empresas podian no ser compatible.

Modelo OSI - TCP/IP

Es un modelo de 7 capas, usando el protocolo TCP/IP, el cual se utilizo para estandarizar todo esto.

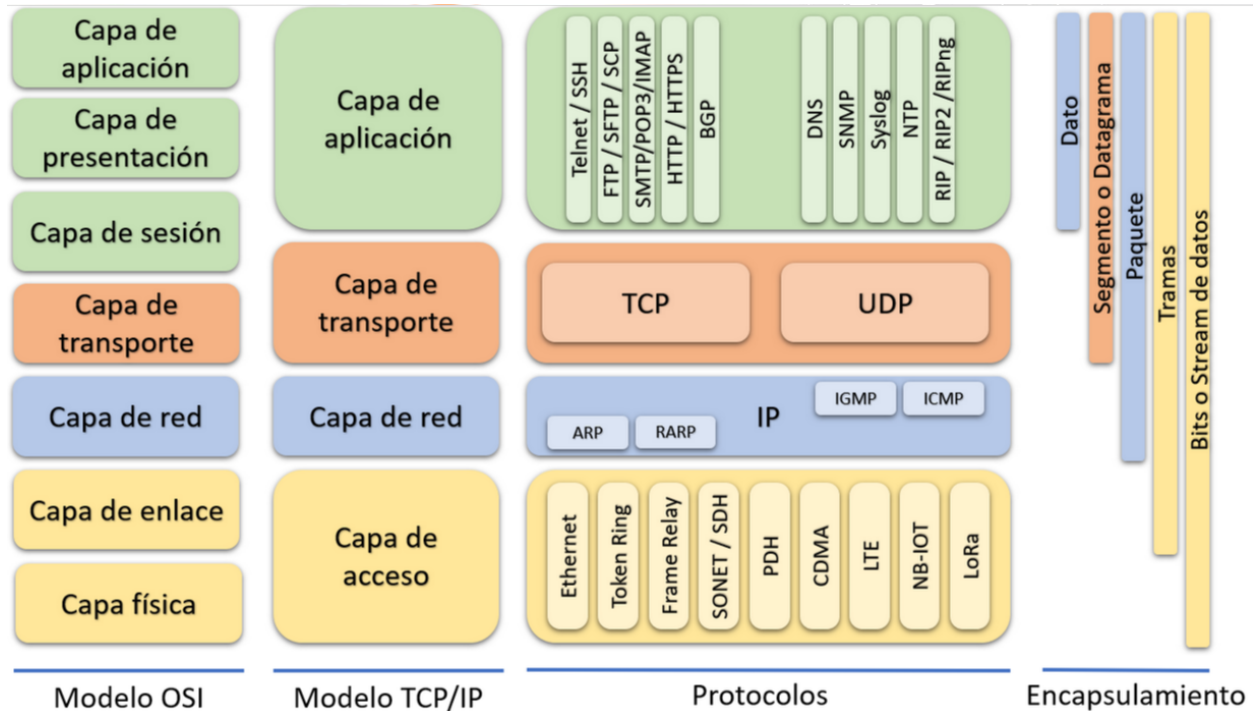


Figure 1: Modelo OSI - TCP/IP

A la capa de red no le importa el motivo, solo le importa hacer llegar un paquete de una computadora a la otra.

Un protocolo es un conjunto de reglas que ambas puntas tienen que seguir

Estos protocolos pueden ser:

- **Confiables:** Confirma si la información fue recibida, utiliza acuse de recibo, reenvía información de ser necesario e informa al nivel superior si no se pudo enviar
- **No confiable:** No puede asegurar si el destinatario recibió o no la información enviable

Capa de enlaces

Tiene varios objetivos:

- Vincular hosts contiguos
- Controlar el acceso al medio físico
- Detectar y/o corregir errores básicos
- Encapsular los paquetes en tramas

Por ejemplo, una topología tipo **bus**. Para evitar colisiones, surgieron dos aparatos:

- **Hub:** Menos inteligente, trabaja en la capa física

- **Switch:** Mas inteligente, trabaja en la capa de enlace. Asocia las MAC de las vocas(va aprendiendo).

Capa de ruteo

Hay que distribuir las IP alrededor del mundo, entonces, se dividio en zonas para que cada una realiza la ditribucion. No se pueden conectar dos host que usan enlaces diferentes, para ello se necesita un router. El router de nuestras casas tiene 2 **interfaces**, pues hay una interfaz por red(WAN y LAN). El router domiciliario es mucho mas que un simple router, pues internamente tiene un switch para el LAN que luego va a una de las interfaces. Sin embargo, a nivel capa de IP, todo esto es una misma red con la excepcion de WAN(red externa). Se podria decir que el switch tambien tiene 2 interfaces mas pues tiene 2.4GHz y 5.8GHz, entonces... tiene 6 interfaces el router? **No**, pues en un switch se llaman vocas(capa enlace), las interfaces son de la capa IP.

En conclusion, vamos a considerar que un router tiene dos interfeceez, todo esto a nivel logico y de seguridad.

Capa de transporte: Puestos

El numero de puerto identifica a los procesos dentro de hosts.

Capa de aplicacion: soporte a aplicaciones

Aca aparecen los servicios mas conocidos: HTTP, POP3, SMTP, DNS, SSH, etc.

Comando que vamos a usar en todas las practicas: `tail -f /var/log/syslog` (-f es de follow)

Serializacion

- Se puede transmitir numeros entre computadoras? Si y no, dependera como codifica el numero(little endian o big endian) una computadora y como lo decodifica la otra. Para solucionar esto, en los protocolos de comunicacion se transmite todo en big endian. La mayoría de las computadoras hoy en dia usan little endian.
- Cuando tenemos que calcular la longitud de un string codificado en unicode, y tiene caracteres especiales, surgen problemas pues los caracteres podrian ocupas mas de un byte.

Unidad 2: HTTP

Antes, para obtener archivos se utilizaba un protocolo llamado **FTP**. En base a las referencias se debia acceder a otro sitio FTP y descargar el/los textos deseados. Surge la necesidad de aplicaciones basadas en **hipertextos**, para no tener que hacer todo manualmente, de ahí surge en HTTP.

HTTP 0.9(1991)

La primer version estaba basado en texto de recuperacion de informacion con un mecanismo request-response. Era contenido estatico y no mantiene el estado. Orientado a la **línea de comando**. Mas adelante se dieron cuenta que tenia potencial comercial, para que sirva para algo mas que papers científicos.

TCP es de texto o binario? Es binario, mientras que HTTP es de texto.

HTTP 1.0(1996)

Ahora necesita ser **browser friendly**, entonces se agregan comandos como GET, POST, HEAD. Además, puede transmitir otras cosas que no sean hipertexto.

HTTP 1.1(1999)

Aparecen cabeceras en las peticiones, se agregaron los métodos PUT, DELETE, TRACE, OPTIONS. Negociación de contenido, soporte de cache, conexiones persistentes. Ahora cada recurso se identifica con una **URI**(Uniform Resource Identifier)

HTTP/2

Pasa a ser un protocolo binario, tiene compresión de headers, multiplexación de recursos, server push, etc. Es mucho más rápido que HTTP 1.1.

Comparación: <http://www.http2demo.io/>

Porque alguien usaría HTTP 1.1? Algun cliente que no soporte HTTP/2 o porque no hay soporte para la tecnología que tenemos que usar.

HTTP/3

Usa el protocolo **QUIC**(por ahí no lleguemos a verlo) como **transporte**, tiene menor latencia, multiplexación mejorada, etc. QUIC es un protocolo de transporte pensado para usar en HTTP pues pide muchos recursos. Desde el punto de vista de la funcionalidad, es de transporte, pero se podría considerar de aplicación.

HTTP/3 no usa TCP, usa QUIC

Recursos

Bloque de información identificado por su URI, puede ser un archivo(físico) o generado por un programa(abstracción). La URI puede ser un URL o un URN. Con URN el archivo puede cambiar de lugar y el servidor se va a encargar de buscarlo, mientras que con URL se pierde.

Que diferencia hay entre una URL y una URI? URI es la clase abstracta para identificar recursos de forma unívoca, mientras que URL y URN son clases concretas.

Los protocolos son públicos y se identifican con RFC(1630, 2396, 2718, 3305, 3986).

URI(RFC 3986 3.3)

`<scheme>://<authority><path>?<query>`

El path termina con el primer ? o # o si no hay más caracteres. Puede ser relativo o absolutos. Si representa una aplicación puede recibir parámetros. El schema identifica el protocolo y la mayoría de los browsers lo agregan por defecto. Pueden ser usadas para acceder recursos, ya sea por medio de URL o URN.

URL

`<scheme>://<user>:<password>@<host>:<port>/<path>?<query>`

Identifica un recurso por su ubicación.

URN

Identifica un recurso por su nombre, no implica que el recurso exista o como acceder a el.

Por ejemplo:

`urn:isbn:0132856204`

Escribimos `www.clarin.com` y el browser se encarga de detectar que es HTTP. Luego el browser averigua la IP(dsp lo vemos) y se conecta mediante TCP al servidor.

En caso(por ejemplo) de AWS, no es factible asignarle una IP publica a cada cliente, lo que se hace es tener todas en el mismo servidor(misma IP publica) y se diferencias las paginas por la URI, entonces, a veces puede no funcionar solicitar una pagina web escribiendo directamente la IP.

Por otro lado, los headers en un request estan delimitados por un espacio(`\n\n`), esto aprovecha que es texto, sino no se podria hacer. Mientras que para saber cuando termina el cuerpo se indica en el header con `content-length`.

Esto funciona perfecto si es estatico, pero surgen problemas si es dinamico.

Request menos utilizados

- PUT
- TRACE
- OPTION
- DELETE

Respuestas

- 1XX: Informacion
- 2XX: Exito
- 3XX: Redireccion
- 4XX: Error en cliente
- 5XX: Error en servidor

La diferencia entre **moved permanently**(301) y **found**(302), a pesar de que ambos solo redireccionan, es que el primero es permanente, no va a cambiar. Por lo tanto, uno es una regla que no va a cambiar, permitiendo que el navegador se lo guarde y lo use para no gastarse en mandar de vuelta el request.

Tipos de informacion

HTTP solo transmite texto, utiliza **MIME** para describir contenido multimedia, cada objeto es etiquetado por el web server. Se define en el header `accept`.

- `text/html`
- `text/plain`
- `video/mp4`
- `image/jpeg`
- `application/pdf`

- etc.

GET vs POST

La idea es que el **GET** sea idempotente. Puede ser cacheado, el browser los mantiene en el historial, pueden ser **bookmarked**, tienen longitud acotada y solo son para recibir datos.

Idempotente es que puedo hacer **GET** todas las veces que pueda y el estado del servidor seguirá siendo el mismo (PUT no es idempotente, puede cambiar el estado en el primer request y en el segundo el estado será otro).

DELETE es idempotente también.

Mientras que el **POST** nunca es cacheado, no se mantienen en el historial del browser, no pueden ser **bookmarked**, no tienen longitud de datos máxima.

Headers HTTP

Como se mencionó anteriormente, finaliza con una línea en blanco.

`<nombre>: <valor asociado>`

Tipos

- Generales
- De solicitudes
- De respuestas
- De contenido

Generalmente se envían en ese orden.

Headers generales

- **Cache-control:** Directivas para cache
- **Connection:** Se definen opciones de conexión
- **Date:** Fecha de creación del mensaje
- **Transfer-Encoding:** Indica el encoding de transferencia
- **Via:** Muestra la lista de intermediarios por los que pasó el mensaje.

Headers de solicitud

- **Accept:** Tipo de contenido aceptado por el cliente
- **Accept-Charset:** Charset (ISO-xxxx, UTF-8) aceptado por el cliente
- **Accept-Encoding:** Encoding (gzip, compress) aceptado por el cliente
- **Expect:** Comportamiento esperado del server frente al request.
- **From:** Email del usuario de la aplicación que generó el request.
- **Host:** Servidor y puerto destino del request.
- **If-Modified-Since:** Condiciona al request a la fecha indicada.
- **Referer:** URL del documento que generó el request

- **User-Agent:** Aplicación que generó el request
- **Upgrade:** solicita que use otro protocolo
- **Range:** solicita un rango (en bytes) del recurso

Referer esta mal escrito(va con *rr*), fue un error y quedo asi :/.

accept es lo que le mandamos al servidor, pero despues el servidor especifica cual te manda en **content-type**.

Headers de respuesta

- **Age:** Estimación en segundos del tiempo que fue generada la respuesta en el server.
- **Connection:** close, keep-alive
- **Location:** URI a redireccionar
- **Retry-After:** Tiempo de delay para reintento
- **Server:** Descripción del software del server.
- **Authorization:** indica que el recurso necesita autorización

Headers de contenido

- **Allow:** Métodos aplicables al recurso.
- **Content-Encoding**
- **Content-Length**
- **Content-Location**
- **Content-MD5**
- **Content-Type**
- **Expires:** Fecha de expiración del recurso.
- **Last-Modified:** Fecha de modificación del recurso

Proxy servers

Foward proxy, es algo que esta en el medio, pueded ser para molestar tambien. Actua como intermediario entre una plicacion cliente y un web server. Puede ser:

- **Explicito:** Lo configuramos nosotros, sabemos de su existencia.
- **Transparente:** A traves de un firewall, no nos damos cuenta que esta.

Tiene varias finalidades:

- Controlar el acceso a determinados sitios o paginas
- Almacenar en cache recientes consultas, haciendo que sean mas rapida pues usamos la red local

Sin embargo, el cache puede causar contenido desactualizado, por lo que el que genera el recurso decide cuanto tiempo dura el cache. Setea el parametro **max-age**.

Proxy reverso

El proxy directo esta del lado del cliente, mientras que el reverso, **reverse proxy** esta del lado del servidor. Redirecciona el trafico internamente a distintos servidores, por ejemplo, **NGINX**.

En el cache del browser/proxy esta **last-modified**, **max-age**, etc.

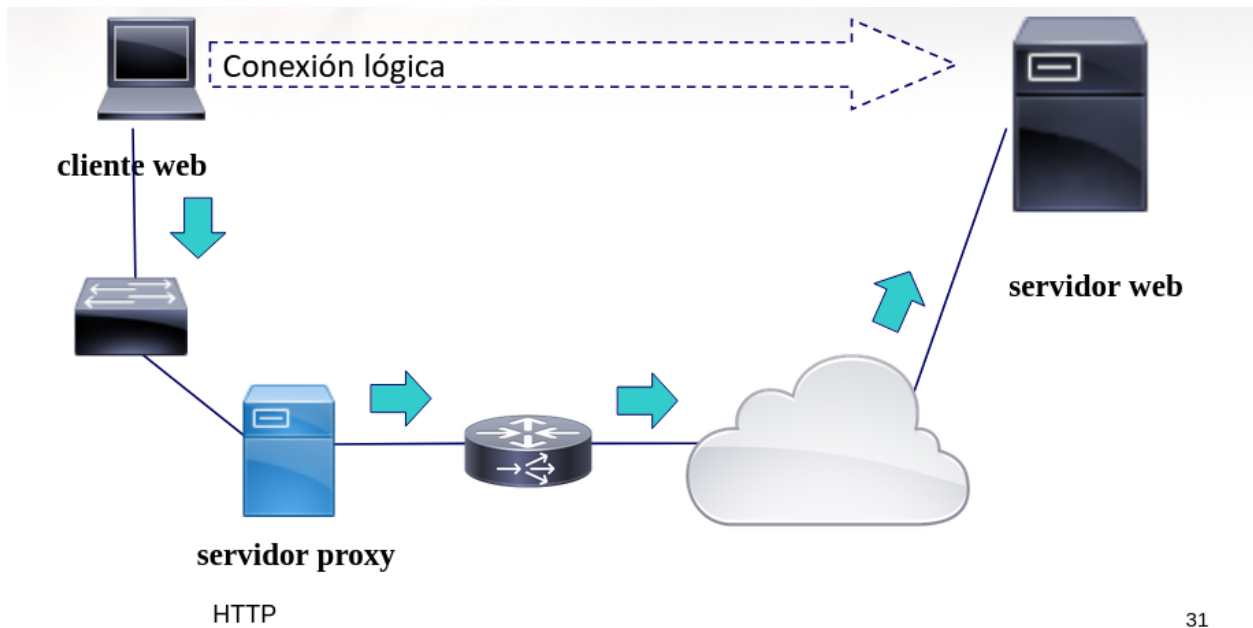


Figure 2: Proxy server

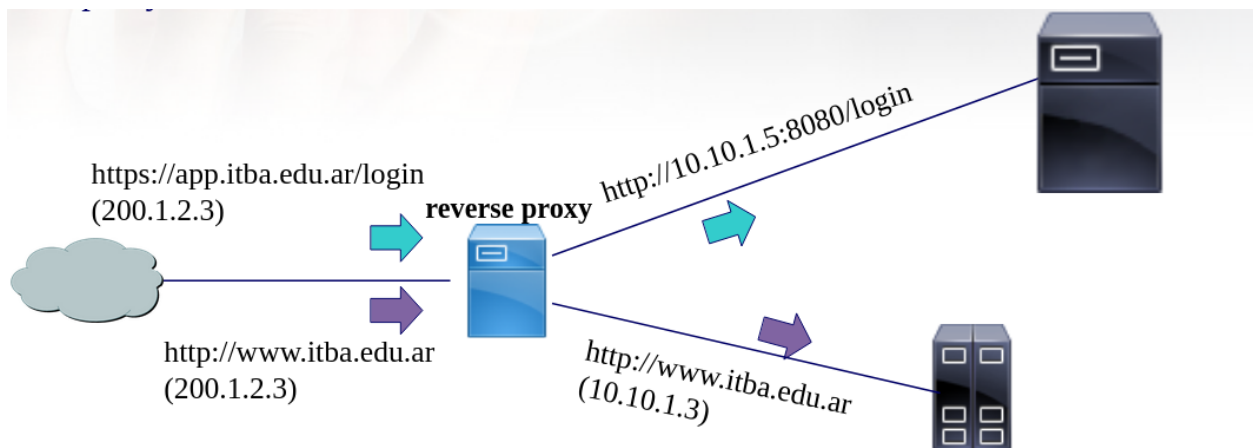


Figure 3: Reverse proxy server

Cookies

La idea es que cuando pida un recurso, en el header esten las cookies. Entonces, cuando el servidor me manda cookies (**set-cookie** res header), a ese servidor en particular, cuando pida un recurso le voy a mandar las cookies(cookie req header).

Es un texto de informacion enviado por el servidor y almacenado en el servidor, se usa para mantener un estado entre el cliente y el servidor. Tiene dos tipos de persistencia:

- Session cookie
- Persistent cookie

Tambien hay **third-party** cookie y **secure** cookie.

Un RFC es un pedido a un estandar, se publica, es revisado por un monton de gente. Si se aprueba pasa a ser un estandar.

Curl

A la hora de descargar archivos:

```
curl <url-archivo> --output <archivo>
```

Si lo detengo a la mitad y vuelvo a correr el comando, curl va a empezar de nuevo. Para que continúe con lo que ya tenía hay que agregar **--continue-at**:

```
curl <url-archivo> --output <archivo-nuevo> --continue-at <archivo>
```

Curl asume que el archivo que le das para continuar es la primer parte de la descarga, caso contrario se unen dos archivos y queda algo raro que no sirve.

Es muy buena la documentacion de **Mozilla** para referenciar.

Unidad 3: Resolucion de nombres

Lo que hace HTTP, primero se crea una conexion mediante TCP, luego se pueden mandar los request. Esta conexion se hace a partir de la IP, entonces como se obtienen las IP a partir del domain? Hay dos formas, una usa DNS.

Resolucion de nombres - DNS

Proceso de traducir un nombre de dominio a una direccion IP.

Un **dominio** es una coleccion de computadores que pueden ser accedidas usando un nombre en comun. Un **nombre de dominio** hace referencia al nombre de multiples hosts que son referenciados colectivamente

Las computadoras de un mismo dominio no tienen que estar en el mismo lugar. No tienen nada que ver.

No todos pueden encargarse de la resolucion de nombre, pero si se pueden tener subdominios. Por ejemplo, solo una organizacion da los .com, pero despues yo puedo tener mis propios subdominios como subdominio.ejemplo.com.

No siempre se tiene que acceder a un DNS, podrian estar localmente en `/etc/hosts`, si se encuentra, se usa eso. Ahi se encuentra `localhost`, pues no conviene que tenga una regla especial, es mejor que se comporte como todos los demas. Tambien sirve si queremos bloquear alguna pagina:

127.0.0.1 chatgpt.com

En examen pueden haber preguntas de que codigos se devuelve en cierta situacion, por ejemplo, si mapeo mi pagina web a `clarin.com` en mi computadora, que pasa cuando el servidor ve que el host del request no es el suyo? Devuelve algun 4XX.

En caso de FQDN, se obtiene su numero de IP del DNS o de `/etc/resolv.conf`. Se pueden especificar:

- `nameserver <direcciones>`
- `domain <nombre>`
- `search <dominios>`
- `sortlist <red\[mascara\]>`
- `option <opcion>`

Los `nameserver` tienen que ya estar especificados por su IP, son los servidores DNS. Cuanto duran las resoluciones en la cache? Depende del `timeout` que te devuelva el servidor DNS junto a la IP.

Linux tiene las utilidades `dig` y `host` para mostrar como la computadora, a partir de un nombre de dominio, encontro la IP(reverse DNS) o al revez.

Si un dia alguna pagina web cambia de IP(cambio de proveedor de internet, cambio de locacion, etc). Entonces, si me quiero conectar, mi computadora va a intentar de acceder a la vieja IP, algo similar va a pasar en el cache de todos los DNS. Entonces, va a haber que notificar a `a.riu.edu.ar` pues es el root server que maneja `edu.ar`, no hace falta notificar al de `ar`.

Como se consiguen los nombres

Es un sistema jerarquico y distribucion para la resolucion de nombres de dominio en direcciones IP(y viceversa). Se lee al revez, empieza la jerarquia mas alta por la derecha. Es **jerarquico y distribuido** porque delega a los de abajo.

El nombre **completo** en realidad es un `.`, es el top-level domain.

Entonces, hay distintos tipos de servidores DNS:

- **Recursivos:** Realizan la consulta completa en nombre del cliente
- **Autorizados:** Contiene la informacion oficial
- **Raiz DNS:** Dominio padre

La distribucion puede ser vista como un arbol invertido donde la raiz hace referencia a servidores de nombre raiz(root name servers).

Los **root name servers** se usan cuando el servidor de nombres local no pueden resolver un nombre. El root name server luego:

1. Contacta a un name server autorizado si no conoce la respuesta
2. Almacena
3. Retorna la respuesta al servidor de nombres local

Hay 13 de estos en total.

Generalmente el **DNS server local** es el router.

Las preguntas y las respuestas de los servidores DNS son UDP.

Cache

Una vez que el servidor de nombres aprende el mapeo, lo almacena en cache.

—

Los servidores raiz DNS son operados por IETF. Cuando alguien obtiene un nombre de dominio es el responsable de mantener, en un servidor DNS primary(antes master), actualizado ese dominio(Completa la idea de lo que ocurre cuando cambio mi IP).

En Unix y Linux DNS es implementado generalmente con **BIND**(Berkeley Internet Name Domain).

Configuracion de servidor DNS

Hay cuatro niveles de configuracion:

- Resolver only systems
- Caching-Only
- Primary(Antes master)
- Secondary(Antes slave)

Registros DNS

DNS es una base de datos distribuida de **resource records**(RR). Los registros son:

- A
- NS
- CNAME
- MX
- AAAA
- SOA
- SRV
- RP
- TXT
- SPF

Los registros asocian el request a distintas cosas, por ejemplo, A devuelve la IP.

SRV es tipo MX, pero mas generico

Formato RR

(name, value, tyype, ttl)

El @ surge para separar el nombre del dominio del nombre del usuario, sino se tornaba confuso.

El tiempo de vida minimo para respuestas negativas se refiere a cuando el servidor tiene que ir a preguntar a otros por un nombre de dominio. Si no lo encuentra, durante las proximas, por ejemplo, 3 horas no volvera intentar a preguntar por el nombre a otros(va a devolver que no lo encontro).

Split-horizon

Un nombre se resuelve con distintas IP dependiendo del origen de la consulta. Por ejemplo, desde el ITBA el servidor de pampero es 10.16.1.103, mientras que desde afuera es 200.5.121.139.

No es si la pregunta se hace desde adentro o desde afuera, sino que donde esta el servidor DNS, si esta afuera me va a dar la IP de afuera por mas que yo este adentro del ITBA.

DNS dinamico

DDNS permite actualizar en forma dinamica un servidor DNS. Introducido en BIND a partir de la version 9. Si cambia la IP, el servidor se va a actualizar automaticamente.

DNS spoofing

Es el arte de lograr que un registro DNS apunte a un IP que no sea el que deberia apuntar. De manera que algun actor malicioso podria crear una copia de la pagina del ITBA y hacer DNS spoofing para que el nombre de dominio del ITBA apunte a su pagina. Esta seria una manera de realizar un ataque de phishing. Por eso es importante preguntar para corroborar la resolucion DNS.

Es comun que muchas empresas sigan usando versiones viejas de software como BIND.

Metodo: Enviar respuesta sin recibir pregunta

Se envia como falsa respuesta el IP de algun nombre de dominio. Se debe verificar que la respuesta corresponda con alguna pregunta. DNS establece que los mensajes de pregunta y respuesta tengan un mismo ID que los identifique.

Metodo: DNS sniffer

Person in the middle(no man in the middle), se obtiene el ID para poder responder con una IP falsa en nombre del servidor DNS.

Metodo: DNS cache poisoning

Inyecta otras resoluciones a una simple pregunta para ensuciar el cache del servidor. Pisa las resoluciones anteriores.

Si pedis el servidor MX por lo general tambien te devuelve A y AAAA pues es muy probable que sino harias otra consulta para averiguar eso.

Notas practica

- Como se sabe si un servidor es root? Basicamente, hay 13 IPs hardcodeadas.
- Los autoritativos nunca hablan con los clientes, ese es el trabajo de los cache, estos si hablan con los autoritativos.
- Si preguntan porque no funciona una zona(porque a x no le aparece?), generalmente se debe a un error boludo, tipo poner mal el serial.

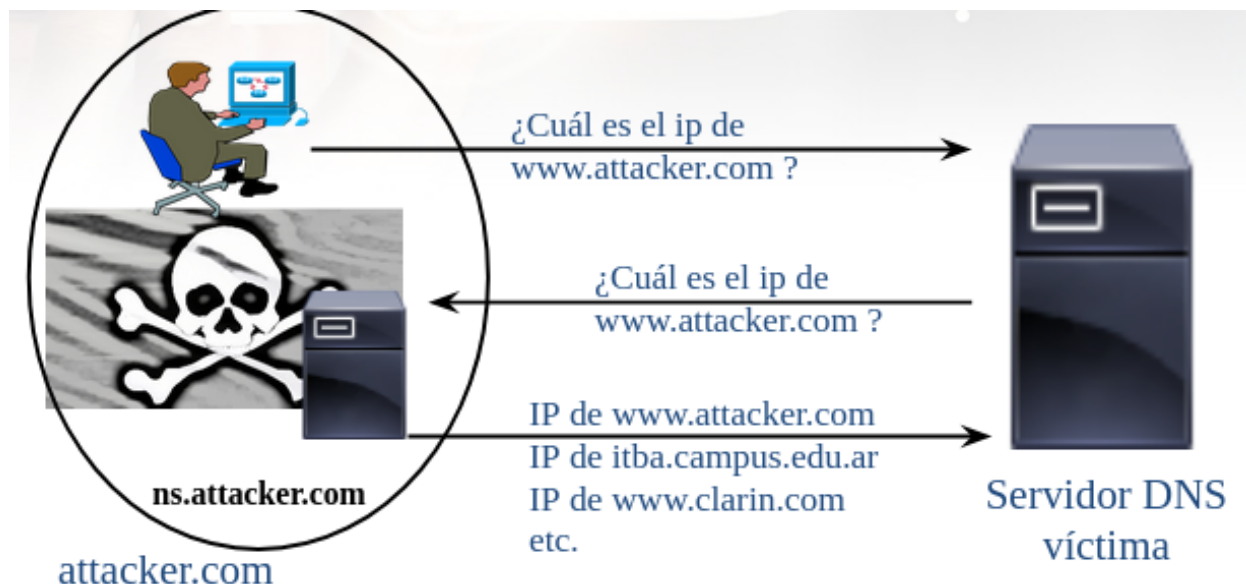


Figure 4: DNS Poisoning

Unidad 4: Correo electronico

Cuando empecé no se considero que iba a ser usado por personas individuales, entonces no había ningún tipo de seguridad.

Antes, en los ambientes Unix un usuario podía mandarle un mensaje a otro de forma asincrónica. Cada usuario tenía una carpeta donde se le dejaban los mensajes, es decir, estaba pensado a los usuarios de un mismo sistema.

Luego, para permitir mandar mails a personas que estén en otros dominios, se creó un protocolo. Así surgió el Simple Mail Transfer Protocol (SMTP) en 1982. Se encarga de dejar en la carpeta de mail de cada usuario el mail enviado a dicho dominio.

Se usa el registro MX de los servidores DNS para resolver los requests de mails. SMTP usa TCP, la transferencia es directa y los mensajes son en US-ASCII. Tiene tres fases:

- Handshaking
- Transferencia de mensajes
- Cierre

Envío de datos binarios

Si no soporta otra cosa que sea US-ASCII (de 7 bits), como se envían binarios por ejemplo? Se codifica antes para que sea un mail válido, luego se decodifica. Se debe codificar como si fuera texto con \n en cada línea u otros.

Similar a HTTP, devuelve códigos para indicar el estado de la transferencia, por ejemplo, 250 es que todo estuvo OK.

Es un protocolo de texto. Se puede usar netcat para crear una conexión y ver esto.

Aca si da lo mismo si uso la IP o el host, a diferencia de HTTP.

Como mandar mail con netcat

Primero vemos cual es el servidor de mail del dominio al que queremos contactar:

```
dig MX <dominio>
```

Luego, creamos una conexion con netcat:

```
netcat <Servidor MX> 25
```

Una vez tengamos una conexion asegurada:

```
EHLO
MAIL FROM <mail>
RCPT TO <mail>
DATA
SUBJECT <subject>
FROM <mail>
\n
<Contenido del mail>
.
EXIT
```

Importante terminar con un .. Es interesante que puedo poner cualquier mail y puedo usar el mail de otro como el que envia. Sin embargo, estos mails suelen terminar en spam pues les falta un monton de informacion.

Si lo pongo al mail del itba me lo va a rechazar porque el servidor de google(que es el que usa el itba) hace una verificacion extra, esta es validar la IP de origen.

Las cadenas de mails se logran mediante un id que tiene cada mail.

Los MX reciben mail, pero hay otros registros que guardan los servidores de mails autorizados. Esto es lo que hacen la mayoria de los servidores para verificar los mails.

Si es algo que no se puede enviar(TO invalido), el campo FROM va vacio para evitar un ciclo infinito, que lo que ocurriria si este es invalido.

Protocolos de entrega final al usuario

- POP3
- IMAP

IMAP evita perder todos los mails pues se guardan en el servidor siempre, caso contrario si yo pierdo o se rompe mi disco duro, voy a perder todos los mails. Hoy en dia practicamente no se usa IMAP, es porque hoy en dia generalmente se usa una aplicacion web para ver los mails(y los mails estan en el servidor). En todo caso la aplicacion web usaria IMAP. Esta practica se conoce como webmail.

Encoding: Base64

Es un metodo de codificacion simple:

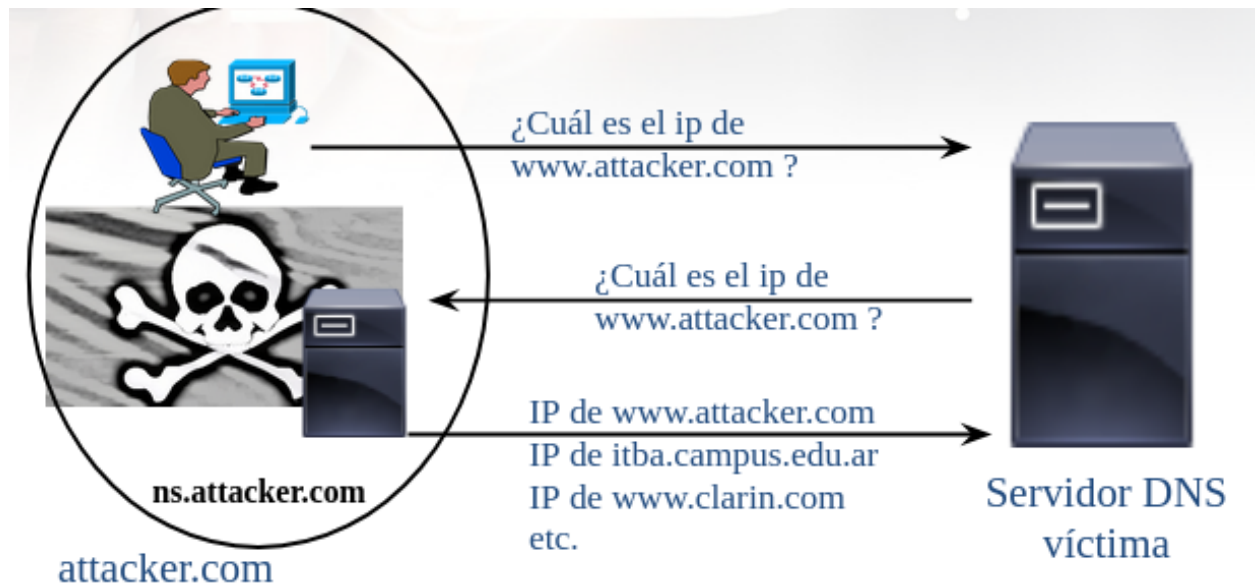


Figure 5: Webmail

- Cada grupo de 3 bytes es codificado como 4 bytes, cada uno conteniendo solo 6 bits de datos.
- Estos son enviados como ASCII de 7 bits.

SPAM

Surge de un sketch de Monty Python.

Como evitarlo:

- Allow list y deny list
- Grey listing (porque antes eran white y black list)

Se clasifican con **filtros bayesianos**. Gray list es que es dudoso, no le acepta ni rechaza la conexión, sino que dice que intente mas tarde. Entonces, si es de SPAM y sigue volviendo a mandar mails entonces para a la lista negra. Una persona haria lo logico de volver a intentar mas tarde.

Tecnicas para autenticar mails

- SPF (Sender Policy Framework)
- DKIM (DomainKey Identified Mail)

SPF

El servidor DNS de un dominio tiene registros para indicar cuales son los servidores autorizados para mandar mails de ese dominio.

DKIM

El servidor que envia el mail verifica que el usuario realmente es el de su dominio, se le agrega un sello para garantizar que no fue alterado.

Unidad 5: TLS/SSL

SSL

Se quiere encriptar datos desde y hacia el servidor, un Web Server en este caso usa SSL. HTTP no ofrecia nada de seguridad o identificacion.

Se conoce como **Secure Sockets Layer**, va antes de la capa de transporte, es la capa que se encargara de encriptacion y seguridad. Se mete en el medio de un protocolo no seguro y el transporte. Es agnostico al protocolo que este encriptando, **HTTPS** no es un protocolo en si, es simplemente HTTP con esta capa extra.

Claves asimetricas

No hay una unica clave que encripta y desencripta. Sino que hay una publica y una privada, un mensaje codificado con la clave publica solo se puede desencriptar con la clave privada.

Entonces es facil, si quiero comunicar a y b, es cuestion de que a tenga la clave publica de b y viceversa.

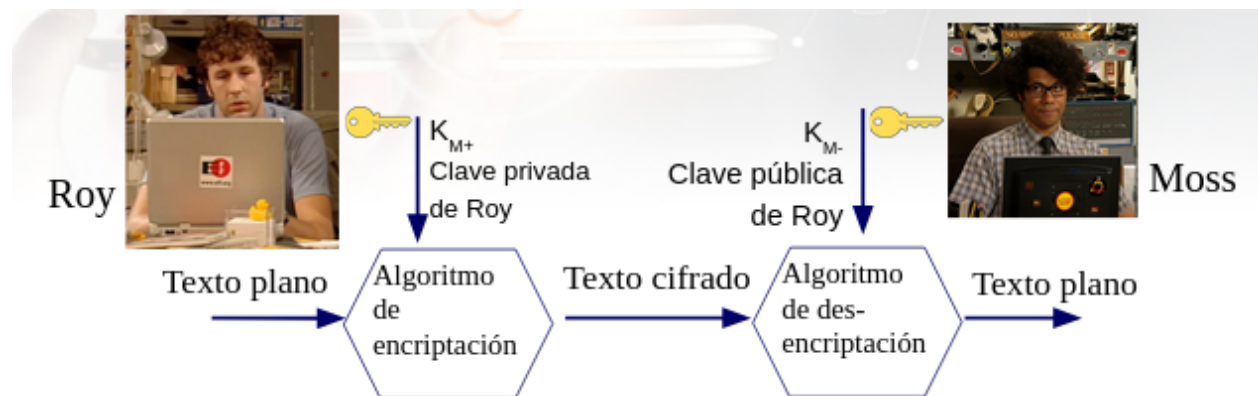


Figure 6: SSL

La primera vez debe ser la persona actual a la que queremos enviar informacion encriptada, pues se transfiere en HTTP simple. Como nos aseguramos que el que este del otro lado sea la persona que creemos que es? Se encripta una passphrase con la clave publica, si el otro responde con la misma clave encriptada con mi clave publica. Si son iguales, entonces esta bien.

En el browser, al usar HTTPS, en vez de preguntarnos cada vez si confiamos en el servidor que esta del otro lado, se usan certificados emitidos por fuentes confiables que el browser usa para chequear si es valida la clave publica.

Se llaman **entidades certificadoras**

En HTTPS se utiliza SSL solamente en la parte inicial para transmitir una clave simetrica y usar esa durante la conexion. Esto se debe a que SSL es bastante pesado.

Se crea una clave simetrica por cada sesion

Tipos de SSL handshake

- Full handshake
- Session resumption

Session resumption acelera el proceso

TLS

Transport Layer Security es lo mismo que SSL, pero inicia la conexion segura de otra manera. Comienza con un **hello** inseguro y luego cambia a una conexion segura. No tiene la necesidad de tener un puerto especifico, se crea la conexion segura en el mismo puerto, no hace falta.

En SSL hay un comando redirect para pasar a otro puerto donde se realizara la conexion segura.

DNS over TLS(DoT)

Usa TCP y se usa para crear una conexion segura con DNS, esto evita tener una persona en el medio y ataques similares.

DNS over HTTPS(DoH)

Es otra manera de tener una conexion mas segura a un servidor DNS, es menos eficiente.

HTTPs y proxy HTTP

Los proxies HTTP se utilizan para cachar contenido, filtrar acceso y mejorar rendimiento. Cuando el trafico es HTTP, el proxy puede inspeccionar, modificar y almacenar las respuestas.

Con HTTPS el trafico esta cifrado entre el cliente y el servidor, lo que impide que un proxy vea el contenido directamente. Entonces, como logran los proxies bloquear sitios o cachear contenido cifrado? Con MITM.

Intercepcion HTTPS con MITM

Se conoce como **Man-in-the-middle**, es la tecnica que usan los proxies para inspeccionar HTTPS. Se coloca entre vos y el servidor, el proxy tendra una conexion HTTPS con vos y otra con el servidor, pasando los paquetes entre los dos y al mismo tiempo tener acceso a eso.

Deteccion

- Verificar el certificado SSL en el navegador.
- Si el emisor no es una entidad oficial podria ser un MITM
- Se pueden usar herramientas como wireshark para detectarlo

Limitaciones

- No funciona si el cliente no confia en el certificado del proxy
- Sitios con HSTS(HTTP Strict Transport Security) pueden rechazar la conexion con certificados no conffiables

- Algunas aplicaciones y navegadores alertan sobre certificados sospechosos
- Let's encrypt usa una query DNS para saber que dominios son validos, los dominios los saca de nuestra configuracion de nginx.

Unidad 6: Protocolos de transporte

DNS puede usar TCP y UDP.

Proveen la comunicacion logica entre dos procesos que corren en distintos hosts. Se ejecutan en las **puntas finales**, hay varios:

- UDP
- TCP
- SCTP(No se usa tanto)

Ninguno, UDP y TCP, ofrecen ni un minimo de demora o latencia, ni minimo de ancho de banda.

Se implementan con sockets, cada proceso tiene un socket. Estan pensados como file descriptors, entonces para hablar con otro host se usa un socket. Es mas, usan la misma tabla que los file descriptors(pues usa los mismos system calls).

Multiplexacion

(en la presentacion de campus hay mas data)

Gran parte de los puertos estan definidos tanto para TCP como para UDP, aunque algunas aplicaciones utilicen solo uno de ellos.

Si no somos root no podemos abrir puertos menores a 255(reservados para aplicaciones publicas).

Demultiplexacion sin conexion

Se usa para saber para que proceso/socket es lo que llego.

- Crear socket con numero de puerto.
- El socket se identifica por el par <IP destino, Puerto destino>
- Cuando el host recibe un segmento UDP -> Dirige el segmento al socket que atiende
- Datagramas con distinto IP origen son atendidos por el mismo socket

UDP

UDP transporta datos de manera no confiable entre hosts.

- No orientado a conexión
- No confiable
- No ofrece verificación de software para la entrega de segmentos
- No reensambla los mensajes entrantes
- No utiliza acuses de recibo (ACK)
- No proporciona control de flujo

Demultiplexacion orientado a conexion

Seria TCP. Cada socket identificado por:

- IP origen
- Puerto origen
- IP destino
- Puerto destino

Host que recibe usa los 4 valores para dirigir el segmento al proceso que corresponde Conexiones simultáneas: c/u con su socket

TCP

Si una maquina tiene dos procesos intentando de conectarse al mismo servidor lo que ocurre es que cambia el puerto en el servidor, manteniendo el puerto de origen.

Repaso conexion confiable

Si el protocolo de red es confiable

- No se alteran bits
- No se pierden paquetes
- Los paquetes llegan en orden

Entonces, la lógica a implementar es muy simple

IP no es confiable.

Si hay errores se complica, pues hay que avisar que llego y que no llego, entonces los algoritmos para **Sender** y **Receiver** no son tan simples. Pero se puede recuperar con **ACK** y con **NAK**, pero se pueden perder esos tambien! Entonces, se manda otro **ACK**, pero el otro podria si haberlo recibido, es por eso que todo paquete debe estar identificado.

Hay algunos controles que se pueden implementar:

- **Stop & wait:** Es trivial, manda **ACK** y se bloquea, pone una alarma en caso de que no llegue y vuelve a mandar **ACK**. Es facil de programar.
- **Pipelining:** Se envian los paquetes y al final se manda un **ACK** diciendo hasta donde recibio. Si se recibio todo termino. Un poco mas complicado, pero mucho mas eficiente. Cada envio tiene una alarma de un tiempo, si no llega el **ACK** hay un error.

TCP no los numera 1, 2, 3... sino que usa bytes, es a nivel de desplazamiento de bytes que esta enviando. El primero no empieza en 0. Se coloca en el campo **Sequence number** del header.

El flag **URG** indica que si hay algun paquete urgente, que los mande primero. Esto no se usa mucho. El flag **RST** corta la conexion de una manera abrupta. En cambio, **FIN** indica que una punta no tiene nada mas que decir, pero se queda escuchando por si la otra punta todavia tiene algo que decir(mas largo que **RST**). Por otro lado, **CRC** es como un checksum, paridad.

TCP ofrece un circuito virtual entre aplicaciones de usuario final.

- Orientado a conexión
- Confiable
- Divide los mensajes salientes en segmentos
- Reensambla los mensajes en el host destino
- Vuelve a enviar lo que no se ha recibido
- Control de flujo (rfc 7323)
- Control de congestión (rfc 5681)

El cliente es el que inicia la conexión, es la única distinción que usa TCP.

En el tercer ACK el cliente ya puede enviar datos.

Si está la conexión y no tengo nada para mandar, de ambos lados se mandan datos **keep-alive**. Son paquetes ACK vacíos para mantener la conexión.

Si se manda el ACK 120 y se envía información **seq=180**, este se guarda en el buffer. Luego, se puede recibir el **seq=120** esperado. Si se vuelve a mandar el 180, se ignora.

El FIN lo puede mandar cualquiera de los dos.

Control de flujo

Si uno manda paquetes y se llena el buffer porque el otro está ocupado con otra cosa, no puede pisar el buffer porque ya mandó el ACK. Tampoco se pueden tirar los paquetes porque estaría mal. Lo que se usa es el size del buffer que se envió en uno de los paquetes (en el campo **window**), entonces no se envían más datos que puedan entrar en el buffer del que va a recibirlos.

El problema es que **window** solo puede especificar hasta un buffer de 64KB, lo cual es inaceptable hoy en día con las velocidades de internet actuales. Lo que se usa son los **options** para negociar y especificar que **window** indica el size con un multiplicador. Es una de las aplicaciones de **options**.

Gracias a esto se evitó armar un TCP nuevo y migrar todo.

Control de congestión

Puede ocurrir que haya un cuello de botella en algún punto de la red, entonces, se pierden paquetes. En teoría, la capa de arriba no debería encargarse de la capa de abajo (en este caso, la capa de red). Sin embargo, se puede tener información de la capa inferior. TCP se puede dar cuenta cuando hay congestión, como hoy en día es poco probable que se pierda un paquete por errores de red, la causa más probable es que hay congestión (y por eso llega el timeout). Entonces, una vez detectado eso, se mandan los paquetes más lento, como todos hacen esto se libera la congestión. Esto se conoce como **AIMD** (Additive Increase Multiplicative Decrease).

La idea es que nadie se haga el vivo, suele ser el caso, pero si ocurre hay forma de detectar si se siguen mandando paquetes a alta velocidad.

Como se decide un valor de **timeout** razonable? Se usa cuando tarda en ir y volver el handshake, de ahí tengo una cota inferior de cuando tardara, uso eso como **timeout**.

Como decidir el **timeout** no aparece en el RFC pues depende de la implementación.

NAPT (aka NAT)

SNAT

Hasta mediados de los 90s para usar internet cada host necesitaba tener una IP publica. Pero rapidamente IPv4 se quedo sin IPs y IPv6 tardara decadas en terminar de implementar globalmente.

Por lo tanto, surgio NAT, de manera que si en una red local se hace un pedido al internet, el router va a registrar la IP local y un puerto. Entonces, cuando sale el pedido al internet es con la IP publica y el puerto asignado.

Sale de la red interna a la externa.

DNAT

Port forwarding, configura NAT para cambiar el puerto de destino. Esto permite implementar balance de carga, teniendo n servidores con una unica IP publica.

Viene de la red externa a la interna.

Firewall

Un firewall opera en el nivel mas bajo de red:

- Evita acceso no autorizado a la Intranet
- Permite definir que trafico entra y sale de nuestra red

Lo vamos a ver mas adelante